



INF8770 : Technologies Multimédias

TP1 : Compression sans perte de textes

Automne 2026

Équipe :

Benjamin Dupuis (Matricule : 2557585)

Gaël Lienard (Matricule : 2489319)

Chargé de travaux pratiques :

Mohamed Aziz Younes

Département de génie informatique et génie logiciel
Polytechnique Montréal

Date de remise : 28 septembre 2026

Table des matières

1	Introduction	2
2	Analyse exploratoire	2
2.1	Question 1 : structure et défis de compression	2
2.2	Question 2 : pistes et choix de méthode	3
3	Implémentation et évaluation	3
3.1	Question 3 : démarche et outils	3
3.2	Question 4 : résultats et discussion	4
4	Conclusion	5

1 Introduction

Dans ce TP, on a décidé de comparer Huffman à deux méthodes sur les trois fichiers du TP (un programme, un texte littéraire en français et un fichier météo au format CSV). On a réfléchi puis essayé un codage prédictif suivi de Golomb, mais les résultats de compression étaient moins bons que Huffman sur tous les fichiers. On a donc essayé puis retenu LZ77, qui bat Huffman sur le code et le CSV mais pas sur le texte littéraire.

2 Analyse exploratoire

2.1 Question 1

Les fréquences des octets sont inégales : l'entropie mesurée va de 4,368 à 4,969 bits par octet, contre 8 bits pour un octet non compressé. Huffman peut donc réduire la taille en donnant des codes courts aux octets fréquents. En revanche, il ne tient pas compte directement des suites de caractères qui se répètent.

Le code contient des espaces (notamment Python à cause des indentations), des mots-clés et des structures récurrentes. Le CSV répète ses en-têtes et certains champs. Le texte contient des mots récurrents, mais peu de longues suites identiques. Ces différences expliquent pourquoi une méthode peut bien fonctionner sur un fichier et moins bien sur un autre.

TABLE 1 – Caractéristiques mesurées des fichiers, en octets, bits/octet et octets par plage.

Fichier	Taille [octets]	H_0 [bits/octet]	Plage moyenne [octets]	Plage max. [octets]
Code Python	3 844	4,368	1,33	78
Littéraire français	5 092	4,422	1,02	2
Météo CSV	26 515	4,969	1,06	3

L'entropie est calculée par $H_0 = -\sum_b p_b \log_2 p_b$. Les plages de même octet sont courtes dans le texte et le CSV. Une compression par plages (RLE) est donc peu adaptée à ces deux fichiers.

2.2 Question 2

Nous avons d'abord essayé une prédiction différentielle suivie du codage de Golomb (vu en cours). On a d'abord essayé cette méthode car un codage différentiel sur des fichiers avec beaucoup de répétitions donne une distribution des octets proche d'une distribution géométrique, ce qui est favorable au codage de Golomb. On a utilisé un codage ZigZag plutôt qu'une approche naïve (modulo 255) pour avoir une meilleure distribution. Malheureusement, cette méthode ne donne pas de meilleurs résultats que Huffman dans nos essais. On suppose que c'est parce que les fichiers contiennent trop de caractères différents et on a une entropie trop grande.

Nous avons ensuite essayé le codage LZ77. Cette méthode remplace une suite déjà rencontrée par une référence, ce qui devrait aider pour le code et les champs répétés du CSV. Le texte littéraire contient moins de longues répétitions exactes, le coût des références risque donc d'y annuler le gain. Dans nos essais, LZ77 a surpassé Huffman sur le code et le CSV, mais pas sur le texte littéraire. On a donc retenu LZ77 pour la suite.

3 Implémentation et évaluation

3.1 Question 3

Le notebook contient Huffman, Golomb prédictif et LZ77, ainsi que leurs essais de décodage. Le code Huffman suit l'exemple vu en cours. Pour LZ77, la fenêtre de recherche fait 32 768 octets et la longueur maximale examinée est de 258 octets. Une suite répétée de trois octets ou plus devient une référence. Sinon, l'octet est écrit tel quel. Les tests du notebook vérifient la reconstruction des trois fichiers. Les outils utilisés sont Python, NumPy, Matplotlib et AnyTree.

La Figure 1 montre les résidus de la prédiction ZigZag. Le CSV a les résidus les plus faibles en moyenne, mais cela ne suffit pas à rendre le codage de Golomb plus efficace que Huffman.

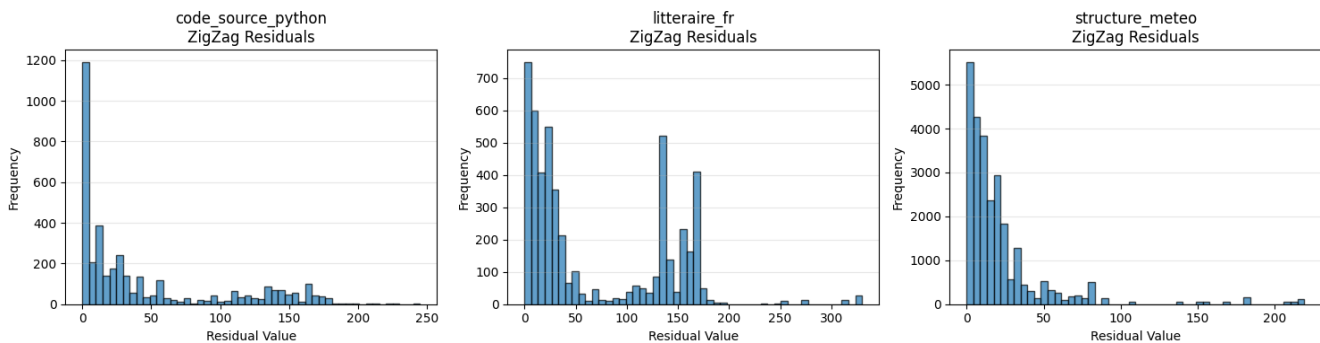


FIGURE 1 – Résidus ZigZag après prédiction. Axe horizontal : valeur du résidu. Axe vertical : nombre d'octets.

3.2 Question 4

Le Tableau 2 reprend les tailles affichées par le notebook. La taille de Huffman comprend une estimation du dictionnaire. La taille de Golomb correspond au nombre de bits divisé par huit. L'en-tête et le remplissage final ne sont pas comptés. Pour LZ77, les quatre octets indiquant le nombre de jetons sont comptés. Les tailles ne comprennent donc pas toutes exactement le même formatage et restent des estimations, mais les écarts observés sont assez grands, pour que ces quelques octets ne changent pas nos interprétations.

TABLE 2 – Tailles estimées [octets] et pourcentage de la taille originale [%].

Fichier	Original [octets]	Huffman [octets, %]	Golomb prédictif [octets, %]	LZ77 [octets, %]
Code Python	3 844	2 383 (62,0)	3 280 (85,3)	1 743 (45,3)
Texte français	5 092	3 078 (60,4)	4 834 (94,9)	3 995 (78,5)
Météo CSV	26 515	16 761 (63,2)	19 830 (74,8)	7 420 (28,0)

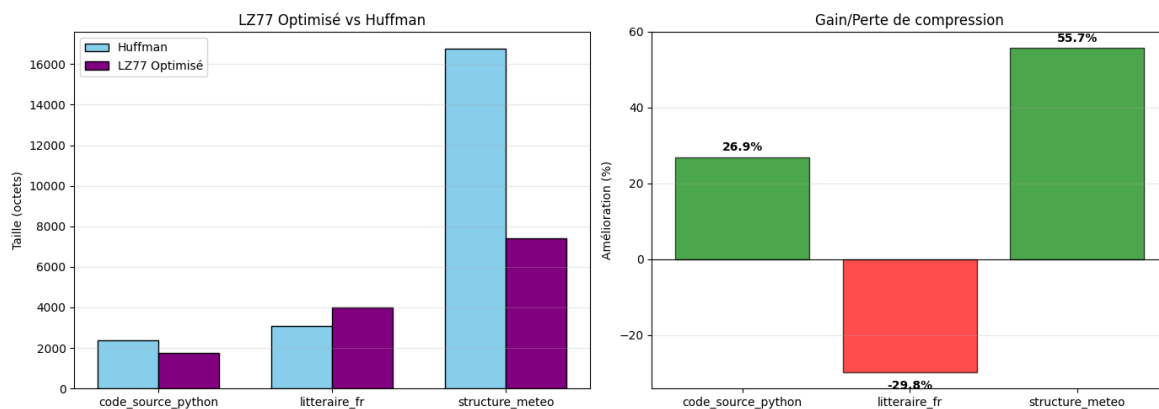


FIGURE 2 – Comparaison Huffman–LZ77 et gain ou perte observé. Les tailles sont en octets, l’amélioration est en pourcentage par rapport à Huffman.

LZ77 améliore la taille par rapport à Huffman de 26,9 % pour le code et de 55,7 % pour le CSV. Pour le texte, il produit un fichier 29,8 % plus grand. Les références LZ77 sont efficaces quand les suites répétées sont assez longues (sur le texte, elles coûtent plus cher que les répétitions ne rapportent). Golomb prédictif reste derrière Huffman sur les trois fichiers, et l’ajout d’un deuxième Huffman ne l’améliore pas (ce qui était prévisible).

4 Conclusion

Notre méthode choisie (LZ77) compresse mieux que Huffman pour le code et le CSV, mais moins bien pour le texte littéraire. La méthode n’atteint donc pas l’objectif initial de surpasser Huffman sur les trois fichiers. Les résultats confirment toutefois que le choix de l’algorithme doit tenir compte des répétitions propres à chaque type de texte.

Enfin, on a également essayé de combiner LZ77 et Huffman, mais les résultats n’étaient pas bon. On pense que c’est un problème dans notre code (que l’on n’a pas réussi à résoudre), car pour comparer on a demandé à ChatGPT de le réimplémenter depuis zéro et les résultats obtenus étaient meilleurs que Huffman sur les trois fichiers.